تمرین سری اول رمزنگاری — "به نام خدا" — یاسمین کریما ۴۰۲۲۴۳۰۹۰

۱- رمز Affine : $y = ek(x) = (ax+b) \bmod 26$ ← $a=7, b=3$

الف)

CRYPTO → 2 17 24 15 19 14

$ek(2) = 14+3 = 17$ — $ek(15) = 4$

$ek(17) = 18$ — $ek(19) = 6$

$ek(24) = 15$ — $ek(14) = 23$

⇒ RSPEGX

ب) برای اینکه تابع معکوس داشته باشد، لازم است $a=7$ به پیمانه 26 معکوس داشته باشد؛ یعنی $\gcd(7,26)=1$ باشد؛ که هست.

$7\times1 \overset{26}{\equiv} 7$ — $7\times9 \equiv 11$

$7\times3 \equiv 21$ — $7\times11 \equiv 25 \equiv -1$

$7\times5 \equiv 9$ — $7\times15 \equiv 27 \equiv 1$

$7\times7 \equiv 23$

پس $a=7$ مجاز است. حالا باید a^{-1} را پیدا کنیم تا تابع رمزگشایی مشخص شود.

$\longrightarrow a^{-1} = 15 \Rightarrow dk(y) = (15y-45) \bmod 26 = (15y+7) \bmod 26$ $(-45 \overset{26}{\equiv} 7)$

⟹ RSPEGX → 17 18 15 4 6 23 →

$dk(17) = 2$ — $dk(4) = 15$

$dk(18) = 17$ — $dk(6) = 19$

$dk(15) = 24$ — $dk(23) = 14$

⇒ CRYPTO √√

۲- $K = (k_1, k_2, k_3, k_4, k_5),\ 0 < k_i < 25$

	A	T	T	A	C
→	L	X	F	O	P
	K	A	T	D	A
→	V	E	F	R	N
	W	N			
→	H	R			

← پیام را به ۵ قسمت تقسیم می‌کنیم:

K_1: $0 + K_1 \equiv 11 \Rightarrow K_1 = 11$

$10 + K_1 \equiv 21$ ✓ V

$23 + K_1 \equiv 7$ ✓ H

K_2: $19 + k_2 \equiv 23 \Rightarrow k_2 = 4 \rightarrow 0 + k_2 \equiv 4$ (E) ✓ ، $13 + k_2 \equiv 17$ (R) ✓

K_3: $19 + k_3 \equiv 5 \equiv 31 \Rightarrow k_3 = 12 \rightarrow 19 + k_3 \equiv 5$ (F) ✓

K_4: $0 + k_4 \equiv 14 \Rightarrow k_4 = 14 \rightarrow 3 + k_4 \equiv 17$ ✓ R

K_5: $2 + k_5 \equiv 15 \Rightarrow k_5 = 13 \rightarrow 0 + k_5 \equiv 13$ ✓ N

۳- الف) حالات ممکن طول کلید: 15، 5، 3.

$d_1 = 48-3 = 45$ — $153-3 = 150$ ⟹ $\gcd(90,45,150) = 15$

$d_2 = 93-3 = 90$

ب) طول کلید $= 5$

$80-10 = 70 \rightarrow \gcd(70,15) = \boxed{5}$

ج) مانند سوال ۲، متن رمزشده را به شکل ماتریس $\frac{5}{k} \times n$ می‌نویسیم، روی هر ستون آن حمله آماری می‌زنیم.

۴-

BALLOON ⟹ BA LL OO N

BA LX LO ON

↓ ↓ ↓ ↓

CD GZ FU QT ⟹ CDGZFUQT

۵-

$K = \begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix}$ $\quad C = x \cdot k \mod 26$

الف) برای اینکه k در پیمانه 26 معکوس داشته باشد، باید $\det \neq 0$ بوده و نسبت به ۲۶ اول باشد.

$\det(K) = 15 - 136 = -121 \overset{26}{\equiv} 9 \Rightarrow 9 \neq 0$ ✓✓

$\gcd(9, 26) = 1$ ✓✓

ب)

$K^{-1} = \underbrace{\frac{1}{\det(K)}}_{\text{معکوس 9 در پیمانه 26}} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \mod 26$

$9 \times 3 \overset{26}{\equiv} 1$ ✓ $\Rightarrow 9^{-1} = 3$

$K^{-1} = 3 \begin{bmatrix} 3 & -8 \\ -17 & 5 \end{bmatrix} = \begin{bmatrix} 9 & 2 \\ 1 & 15 \end{bmatrix}$

ج)

HELP ⟹ 7 4 11 15 $= \begin{bmatrix} 7 & 4 \\ 11 & 15 \end{bmatrix}$

encryption $\Rightarrow C = \begin{bmatrix} 7 & 4 \\ 11 & 15 \end{bmatrix} \overset{K}{\begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix}} = \begin{bmatrix} 25 & 16 \\ 24 & 3 \end{bmatrix} \Rightarrow$ ZQYD

decryption $\Rightarrow P = \begin{bmatrix} 25 & 16 \\ 24 & 3 \end{bmatrix} \overset{K^{-1}}{\begin{bmatrix} 9 & 2 \\ 1 & 15 \end{bmatrix}} = \begin{bmatrix} 7 & 4 \\ 11 & 15 \end{bmatrix} \Rightarrow$ HELP ✓✓

۶-

$P \in \{a, b\}$, $C \in \{1, 2, 3\}$ $\quad K \in \{k_1, k_2, k_3\}$ $\quad P_r[a] = 1/4$ $\quad P_r[b] = 3/4$

$P = 1/3$

الف)

$P_r[y=1] = P_r[k=k_1 \text{ and } x=a] + P_r[k=k_3 \text{ and } x=b] = 1/3 \times 1/4 + 1/3 \times 3/4 = \frac{4}{12} = \frac{1}{3}$

$P_r[y=2] = P_r[k=k_2 \text{ and } x=a] + P_r[k=k_1 \text{ and } x=b] = 1/3 \times 1/4 + 1/3 \times 3/4 = 1/3$

$P_r[y=3] = P_r[k=k_3 \text{ and } x=a] + P_r[k=k_2 \text{ and } x=b] = \sim \quad \sim = 1/3$

ب)

$P_r[x=a \mid y=j] \Rightarrow P(A|B) = \frac{P(B|A)P(A)}{P(B)} = \frac{P(A \cap B)}{P(B)}$

$P(y=1 \mid x=a) = P(k_1)$

$\Rightarrow P_r[x=a \mid y=1] = \frac{1/3 \times 1/4}{1/3} = 1/4$

$$P_r[x=a \mid y=2] = \frac{1/3 \times 1/4}{1/3} = 1/4$$

$$P_r[x=a \mid y=3] = \frac{1/3 \times 1/4}{1/3} = 1/4$$

پ) احتمال رخ دادن $x=a$ به‌ازای هر y ، همان احتمال اولیه درزبان باقی مانده ؛ بنابراین مهاجم با دیدن متن رمزشده (y‌ها) نمی‌تواند الگوی خاصی کشف کند ، بخصوص چون 2 و 3 به صورت گمراه‌کننده جای a و b قرار گرفته اند .

۷- معیارهای Golomb

$T = 7$

100 10 11

در تناوب

100101111001 0111.... (دوره تناوب نرد) $4-3=1$ ✓✓ : اختلاف تعداد بیت های 0 و 1 حداقل باشد

00 10 111 → ران 4

— $\frac{1}{2^i}$ تا از run ها، طول نداشته باشند :

طول	محاسبه تعداد	تعداد واقعی
1	$1/2 \times 4 = 2$	2 ✓✓
2	$\frac{1}{4} \times 4 = 1$	1 ✓✓
3	×	1 →

$T = 2^n - 1 = 2^3 - 1 = 7$

قاعده‌ی تقریبا 1 ران : طول n شامل 1 ها ✓✓

تابع خودهمبستگی

$K(0) = 7 - 0 = 7 = N = K(7)$

$K(1) = 3 - 4 = -1 = K(6)$ ← 1001011 / 1001 0111

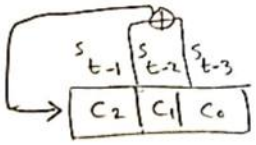
$K(2) = 3 - 4 = -1 = K(5)$ ← 1001011 / 0101110

$K(3) = 3 - 4 = -1 = K(4)$ ← 1001011 / 1011100

⇐ بله ، این دنباله معیار سوم ملومب را دارد و pn-sequence حساب می‌شود .

---

۸ - الف)



$S(t) = C_2 S(t-1) + C_1 S(t-2) + C_0 S(t-3)$

1 s_{t-3}
0 s_{t-2}
0 s_{t-1}
1 s_t
0 s_{t+1}
1 s_{t+2}

$$\Rightarrow \begin{bmatrix} s_t \\ s_{t+1} \\ s_{t+2} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} C_0 \\ C_1 \\ C_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \Rightarrow \begin{cases} C_0 = 1 \\ C_2 = 0 \\ C_1 = 1 \end{cases}$$

$f(x) = x^3 + x + 1$

$S_{t+3} = S_{t+1} \oplus S_t = 1$

$S_{t+4} = S_{t+2} \oplus S_{t+1} = 1$

ب) ۲ حدس ← بیت ششم ۰ است یا ۱ ، پس تابع فیدبک را می‌سازد و بررسی می‌کند آیا با آن کلید می‌تواند بقیه متن را رمزگشایی کند؟ اگر به متن معنادار رسید، حدسش درست بوده و کلید را پیدا کرده است.

پ) چون این ضعف به خاطر ساختار خطی تابع فیدبک به‌وجود آمده. معیارهای کلومب، بررسی آماری می‌کنند تا رمزنگاری تا حد خوبی متقاعدی باشد، اما غیرخطی بودن عملیات را تضمین نمی‌کند.

۹-

$C_1 = 11010110$ $\quad C_2 = 10011100$

$M_1 = 01101100$ $\quad \Rightarrow C_1 = M_1 \oplus K \;,\; C_2 = M_2 \oplus K$

الف)

$C_1 = M_1 \oplus K \Rightarrow K = C_1 \oplus M_1 = 10111010$

$C_2 = M_2 \oplus K \Rightarrow M_2 = K_1 \oplus C_2 = 00100110$

ب)

$C_1 \oplus C_2 = 01001010$

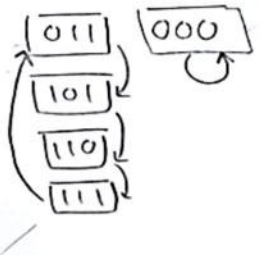
$M_1 \oplus M_2 = 01001010$ $\quad \updownarrow = \surd\surd$

پ) وقتی دوره تناوب کوتاه باشد، قابل رویت است ← مهاجم با XOR کردن C، ارتباط متن اصلی را کشف می‌کند.
همچنین اگر بتواند بخشی از متن اصلی را حدس بزند (مثل الف) آنگاه کلید لو می‌رود!

۱۰-

$f = S_0 \oplus (S_1 \cdot S_2)$

الف)



ب) خیر ندارد.

پ) دو دوره تناوب برایش هست یکی به طول 3 و دیگری به طول 4. آن 000 هم بحساب نمی‌آوریم. اگر هرکدام از اعضای تناوب اول یا دوم استفاده شوند، حلقه در همان‌ها گیر می‌کند.
این درحالی است که می‌توانست دوره تناوبی به طول حداکثر 7 داشته باشد!

ت) خیر، مهم‌ترین دلیل دوره تناوب کم است؛ دلیل دیگر وابستگی به مقدار اولیه است؛ اگر از دوره ۱ مقدار بگیرد درهمان ۳ حالت می‌ماند و اگر از دوره ۲ مقدار بگیرد، درهمان ۳ حالت می‌ماند؛ درحالی که می‌توانست تا 7 حالت را بگیرد...

به نام خدا

گزارش تمرین اول رمزنگاری، بخش کدی

یاسمین کریما ۴۰۲۲۴۳۰۹۰

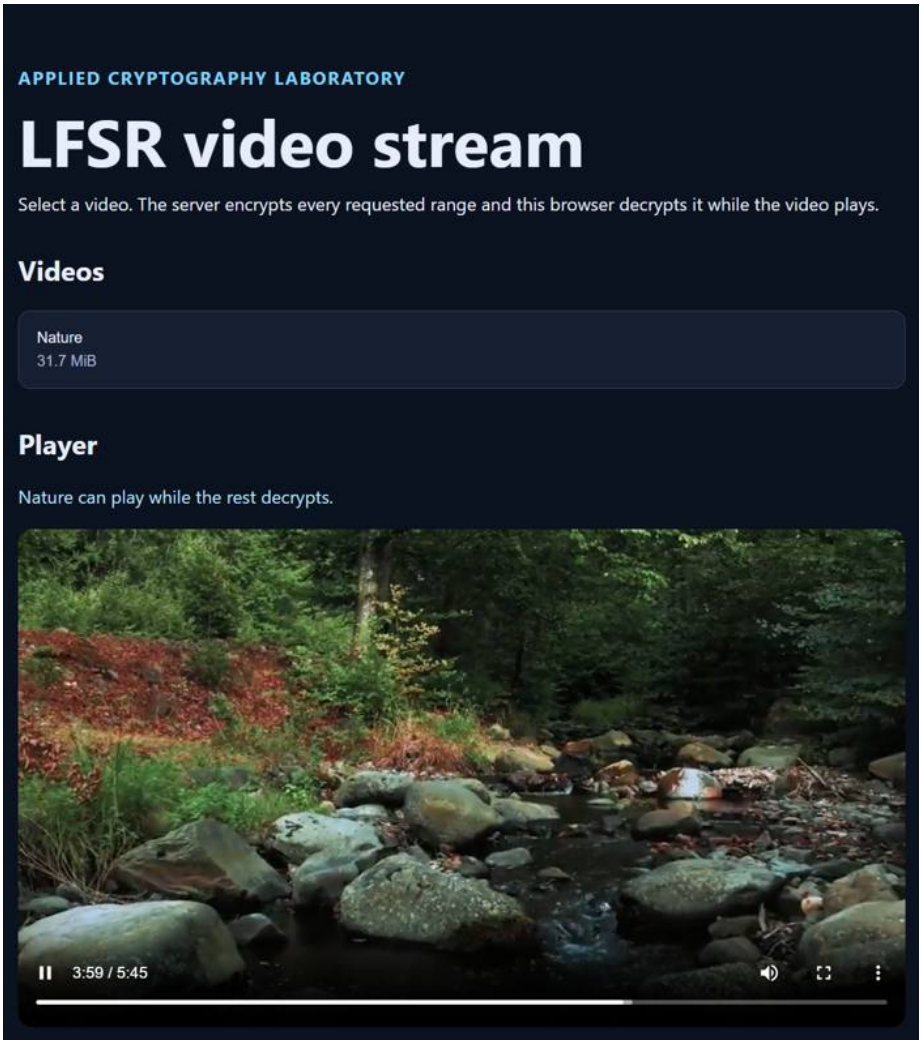
لینک گیتهاب تمرین:

https://github.com/Yasamin-karima/secure_video_steam

طراحی و پیاده‌سازی سیستم

در این پروژه ما بخش رمزنگاری ویدیو در حال استریم را توسط رمزهای جریانی به صورت عملی انجام دادیم، سپس به صورت عملی بررسی کردیم که پخش به درستی کار میکند، در صورتی که فیلم را جلو بزنیم، کل فیلم دانلود نمیشود؛ بخشی از کلید برای سنکرون شدن ساخته میشود ولی استفاده نمیشود.

نتیجه اجرا در مرورگر:



ساختار رجیسترها و توابع پیاده سازی شده

همانطور که در صورت پروژه از ما خواسته شده بود، یک تابع نوشتیم که بیت output را با توجه تابع polynomial داده شده محاسبه میکند. البته نکته آنجا بود که توان x^16 نشان‌دهنده خروجی ثبات یعنی کم‌ارزش‌ترین بیت (LSB یا همان x[0]) در سمت راست بود و فیدبک حاصل در پرارزش‌ترین بیت در سمت چپ وارد میشد.

سپس تابع دیگر keystream را نوشتیم که با توجه به بیت خروجی و آفست بایت مدنظر کلید تولید میکرد. منظور از آفست آن است که اگر کاربر برای مثال ۱۰ دقیقه فیلم را جلو بزند، در اصل تعدادی بایت از فیلم رد شده، پس جریان کلید تولیدی برای encrypt و decrypt باید هماهنگ باشد. برای اینکار از آفست برای نشان دادن تعداد بیت های رد شده استفاده میشود و در تابع نیز برای رد کردن، next_bit صدا زده میشود و استیت بعدی

LFSR ساخته میشود، اما نگه داشته نمیشود. سپس وقتی به آفست مدنظر رسیدیم، یک جریان از بیت های خروجی تولید میکنیم که کلید را میسازند.

در تابع بعدی نیز نوشتیم که چگونه با داشتن آفست ویدیو و بایت های آن، با استفاده از تابع keystream، داده را رمز میکنیم.

سپس همه اینها در جاوااسکریپت پیاده شد.

سپس دو تابع غیر خطی نوشتیم که یکی از عامل غیرخطی AND استفاده کرده بود و دیگری از ترکیب چند LFSR.

ارزیابی کارایی و تست سرعت

دستور evaluate را روی سیستم اجرا کردیم تا نتایج تست ها را ببینیم.

مقایسه عملکرد در بخش بنچ‌مارک

در تست بنچ‌مارک کلی استریم با داده ۱۳۱۰۷۲ بایتی (در قالب ۲ قطعه):

- زمان سپری‌شده کلی: ۰.۵۷۶ ثانیه
- زمان تاخیر متوسط هر قطعه: ۲۸۸.۰۹ میلی‌ثانیه
- نرخ رمزکردن (Throughput): ۱.۸۲ مگابیت بر ثانیه (Mbps)
- بیت‌ریت فرضی ویدیو: ۲.۰ مگابیت بر ثانیه
- نسبت Headroom: مقدار ۰.۹۰۹

آیا سرعت کافی است و معیار چیست؟ معیار ما این است که نرخ تولید دنباله کلید باید بیشتر از نرخ مصرف بیت‌ریت ویدیو باشد (یعنی نسبت بزرگتر یا مساوی ۱ باشد). اینجا نرخ ۱.۸۲ در مقایسه با ۲ مگابیت بر ثانیه باعث شده نسبت ۰.۹۱ شود؛ یعنی در پایتون خام سرعت تولید کلید کمی از بیت‌ریت ۲ مگابیت بر ثانیه عقب‌تر مانده و اگر کش یا بافر کلاینت نباشد ممکن است در پخش‌های بیت‌ریت بالا تاخیر بیفتد، اما برای ویدیوهای با کیفیت استاندارد یا وب کفایت میکند.

مقایسه سرعت سه روش (generator_comparison)

در مقایسه سرعت بین سه مولد:

- مولد Geffe: سرعت ۰.۷۳۵ مگابیت بر ثانیه
- مولد LFSR ساده: سرعت ۰.۶۲۳ مگابیت بر ثانیه
- مولد NLFSR: سرعت ۰.۴۳۸ مگابیت بر ثانیه

در اینجا مولد NLFSR به خاطر محاسبه اضافه جمله ضربی کندتر شده، اما گفه به خاطر نوع گام زدن و ماهیت رجیسترهایش توانسته خروجی بهتری ثبت کند، هرچند که در حالت کلی هر سه در محدوده چندصد کیلوبیت تا نزدیک ۱ مگابیت بر ثانیه کار میکنند.

از نظر تئوری، مولد گفه به دلیل اجرای ۳ رجیستر مجزا به نظر میرسد باید سربار محاسباتی بیشتری داشته باشد؛ اما در بنچ‌مارک تجربی، سرعت آن (۰.۷۳۵ مگابیت بر ثانیه) اندکی از LFSR ساده (۰.۶۲۳ مگابیت بر ثانیه) بیشتر ثبت شد. این موضوع نشان می‌دهد در کدهای سطح بالای پایتون، سربار فراخوانی‌های مکرر توابع (Function Call Overhead) و حلقه‌های درونی اثر به مراتب بزرگ‌تری نسبت به تعداد عملیات جبری بیتی دارند.

تحلیل آزمون‌های آماری NIST (پکیج nistrng)

دنباله‌های تولیدی را به ابزار NIST دادیم تا ببینیم از نظر آماری چقدر شبیه به دنباله تصادفی رفتار میکنند:

نتایج تست‌ها برای سه مولد

- ۱. تست Monobit (Frequency):
- هدف آزمون: بررسی تعادل نسبت تعداد ۰ و ۱ در کل دنباله. دنباله تصادفی ایده‌آل باید سهم برابری از ۰ و ۱ داشته باشد.
- تفسیر نتایج: در LFSR و Geffe هر دو امتیاز بسیار بالا (حدود ۰.۹۸) کسب کردند چون ساختار ثبات با چندجمله‌ای بدوی تعادل کامل (قانون اول گلومب) ایجاد می‌کند و ترکیب گفه هم به دلیل تقارن نسبی مالتی‌پلکسر توزیع متوازنی می‌دهد. اما در NLFSR این نمره به ۰.۱۷ افت کرده است؛ علت این است که جمله غیرخطی (bit 1 AND bit 7) فقط در ۲۵٪ مواقع ۱ می‌شود و این ضرب نامتقارن، تعادل بی‌نقص تعداد ۰ و ۱ را کمی به هم زده و تمایل آماری ایجاد کرده است.

۲. تست Frequency within a Block:

- هدف آزمون: بررسی نسبت تعداد ۰ و ۱ها درون بلوک‌های مجزا و خرد، تا اطمینان حاصل شود که بیت‌ها در طول توالی به طور محلی نیز توزیع یکنواختی دارند.
- تفسیر نتایج: هر سه مولد در این تست پاس شدند؛ NLFSR امتیاز کامل ۱.۰ را ثبت کرده در حالی که Geffe به مرز ۰.۰۴ رسیده است. این افت امتیاز در گفه به دلیل همبستگی ۷۵ درصدی خروجی با ثبات‌های زیرین است که در بلوک‌های کوچک باعث پدیدار شدن نوسانات و عدم تقارن می‌شود.

۳. تست Runs و Longest Run:

- هدف آزمون: آزمون Runs تعداد دسته‌ها و تغییر وضعیت‌های پشت‌سرهم از ۰ به ۱ و برعکس را می‌سنجد (قانون دوم گلومب). آزمون Longest Run بررسی می‌کند که آیا طولانی‌ترین توالی پیوسته از ۱ها درون یک بلوک مطابق توزیع تئوری تصادفی است یا خیر.
- تفسیر نتایج: آزمون Runs در هر سه مولد پاس شد (در LFSR و Geffe امتیاز نزدیک به ۰.۹۸ و در NLFSR حدود ۰.۰۲). اما در Longest Run، مولد LFSR با امتیاز ۰.۰۵ در مرز قبولی ماند ولی NLFSR و Geffe رد شدند (مقادیر بسیار نزدیک به صفر). علت آن است که وجود روابط ضربی در NLFSR و کنترلگر در گفه باعث ایجاد خوشه‌های بزرگتر یا فشرده‌تری از ۱های پیوسته در برخی بلوک‌ها شده که با مدل توزیع نرمال استاندارد همخوانی ندارد.

۴. تست‌های ساختاری و الگوها (Template Matching و Binary Matrix Rank):

- هدف آزمون‌ها: تست Binary Matrix Rank وابستگی خطی بین زیررشته‌های بیت‌ها را در قالب ماتریس می‌سنجد. تست Template Matching جستجوی الگوهای از پیش تعیین‌شده و پیش‌بینی‌پذیر را انجام می‌دهد.
- تفسیر نتایج: تست رتبه ماتریسی (Binary Matrix Rank) در LFSR ساده کاملاً رد شد (score=0.0)، اما در NLFSR (امتیاز ۰.۸۸) و Geffe (امتیاز ۰.۳۵) پاس شد! دلیل واضح این تفاوت این است که LFSR یک سیستم کاملاً خطی با مرتبه محدود است و ردیف‌های ماتریس ساخته‌شده از بیت‌های آن وابستگی خطی قطعی دارند و رتبه ماتریس افت شدید پیدا می‌کند. اما اضافه شدن عملیات غیرخطی (ضرب بیتی در NLFSR و مالتی‌پلکسر در گفه) این وابستگی خطی مستقیم را می‌شکند و رتبه ماتریس را تصادفی نشان می‌دهد. همچنین در Template Matching، وجود تقارن خطی در LFSR ساده باعث کشف الگو و رد آزمون شد، در حالی که لایه غیرخطی در دو مولد دیگر توانست آزمون عدم تطابق الگو را پاس کند.

۵. آزمون Maurer's Universal:

- هدف آزمون: ارزیابی میزان فشردگی‌پذیری دنباله بدون از دست رفتن اطلاعات؛ دنباله تصادفی واقعی نباید قابلیت فشرده‌سازی بالایی داشته باشد.
- تفسیر نتایج: LFSR ساده با امتیاز ۰.۰۳۵ پاس شد اما NLFSR و گفه رد شدند. چون پریود و پیچیدگی ساختار این دو مولد روی حجم محدود داده رفتارهای خوشه‌ای نامتعارفی نشان داد.

بررسی علت رد شدن آزمون‌ها

۶. آزمون‌های ردشده در تمام مولدها (Linear Complexity, Serial, Approximate Entropy, DFT, Cumulative Sums, Random Excursion):

دلیل قطعی: تمام این آزمون‌ها وابستگی شدید به طول کلید و پریود بسیار طولانی دارند. در ثبات ۱۶ بیتی، دوره تناوب حداکثر ۶۵۵۳۵ است و دنباله بعد از مدت کوتاهی وارد چرخه تکرار می‌شود. تست Linear Complexity بلافاصله به دلیل پایین بودن مرتبه خطی شکست می‌خورد؛ تست DFT قله‌های هارمونیک و نوسانات دوره‌ای تکرارشونده را به سرعت آشکار می‌کند؛ و آزمون‌های Serial و Approximate Entropy همبستگی‌های فرکانسی میان n-gramها را به دلیل حجم کوچک حالت (State Space) ثبت می‌کنند.

تحلیل امنیتی و پاسخ به سوالات

چرا قبولی در آزمون‌های آماری پیش‌بینی‌ناپذیری LFSR را اثبات نمیکند؟

همانطور که در نتایج هم دیدیم، LFSR در آزمون‌های پایه مثل نسبت صفر و یک‌ها (Monobit) و Runs نمره قبولی خیلی بالایی گرفت چون ویژگی‌های آماری گلومب را دارد. اما به علت خطی بودن امنیت کامل ندارد. طول ثبات ما ۱۶ است، یعنی پیچیدگی خطی آن فقط ۱۶ است. طبق الگوریتم Berlekamp-Massey، یک مهاجم اگر فقط ۲n بیت یعنی ۳۲ بیت از خروجی کلید را بداند، میتواند با حل دستگاه معادلات خطی، کل چندجمله‌ای feedback و تمام وضعیت‌های بعدی ثبات را پیدا کند و کلیدهای بعدی را پیش‌بینی کند. حتی اگر 2n بیت را نداشته باشد، با brute force میتواند به آن هم دست پیدا کند.

استفاده دوباره از یک seed چه اطلاعاتی درباره دو متن آشکار میکند؟

در رمزهای جریانی مثل این پروژه اگر از یک seed دوباره استفاده کنیم (حمله Two-Time Pad)، کلید تولید شده برای هر دو ویدیو یا دو قطعه کاملا یکسان خواهد بود:

C1 = M1 ^ K

C2 = M2 ^ K

اگر مهاجم این دو متن رمز را با هم XOR کند:

C1 ^ C2 = (M1 ^ K) ^ (M2 ^ K) = M1 ^ M2

یعنی کلید K کاملا حذف میشود و XOR دو پیام اصلی به دست می‌آید. برای فایل هایی از جنس عکس و ویدیو،داشتن این اطلاعات مفید است و موجب نشت اطلاعات میشود.

اثر جمله بازخورد غیرخطی و تابع ترکیب گفه

- در NLFSR یک جمله ضربی (bit 1 AND bit 7) به فیدبک اضافه کردیم. این کار باعث میشود سیستم از حالت خطی خالص خارج شود و الگوریتم Berlekamp-Massey نتواند با ۳۲ بیت آن را حل کند.
- در گفه (Geffe) از سه ثبات استفاده شد که اولی مثل یک انتخاب‌گر یا مالتی‌پلکسر بین دو تای دیگر عمل میکند تا پیچیدگی جبری بالا برود.

ضعف هم‌بستگی مولد گفه و محدودیت حالت و دوره NLFSR کوچک

- ضعف هم‌بستگی گفه (Correlation Attack): با اینکه تابع گفه غیرخطی است، ولی استقلال آماری ندارد. خروجی در ۷۵ درصد مواقع با خروجی ثبات دوم و در ۷۵ درصد مواقع با خروجی ثبات سوم همبستگی دارد (P=0.75). این ضعف اجازه میدهد مهاجم به جای حمله به کل فضای ۲ به توان ۴۸، ثبات‌ها را جداگانه با شمارش همبستگی بشکند و سیستم را با تقسیم و حل کرک کند.
- محدودیت NLFSR: در LFSR خطی چون از چندجمله‌ای primitive استفاده میکردیم مطمئن بودیم دوره تناوب ما ماکسیمم (1 - 2^16) است. اما در NLFSR اضافه کردن AND تضمینی برای دوره ماکسیمم نمیدهد و ممکن است فضای حالت به چرخه‌های خیلی کوتاه چند ده بایتی تقسیم شود و کلید زود تکرار شود. همچنین چون رجیستر ۱۶ بیتی خیلی کوچک است، حتی بدون حمله تحلیلی با Brute-force در چند ثانیه تمام حالت‌هایش میتواند تست شود.